

part E. Functions

Function Parameters

Take the following `sing` function that takes in a `song` parameter and logs it:

```
function sing(song) {  
  console.log(`Singing: ${song}!`);  
}
```

What value type did the developer who wrote the `sing` function intend for the `song` parameter to be provided with?

Is it a `string`? Is it an object with an overridden `toString()` method? Is this code buggy? *Who knows?!*

Without explicit type information declared, we may never know—TypeScript will consider it to be the `any` type, meaning the parameter’s type could be anything.

As with variables, TypeScript allows you to declare the type of function parameters with a type annotation. Now we can use a `: string` to tell TypeScript that the `song` parameter is of type `string`:

```
function sing(song: string) {  
  console.log(`Singing: ${song}!`);  
}
```

Much better: now we know what type `song` is meant to be!

Note that you don’t need to add proper type annotations to function parameters for your code to be valid TypeScript syntax. TypeScript might yell at you with type errors, but the emitted JavaScript will still run. The previous code snippet missing a type declaration on the `song` parameter will still convert from TypeScript to JavaScript. will cover how to configure TypeScript’s complaints about parameters that are implicitly of type `any` the way `song` is.

Required Parameters

Unlike JavaScript, which allows functions to be called with any number of arguments, TypeScript assumes that all parameters declared on a function are required. If a function is called with a wrong number of arguments, TypeScript will protest in the form of a type error. TypeScript's argument counting will come into play if a function is called with either too few or too many arguments.

This `singTwo` function requires two parameters, so passing one argument and passing three arguments are both not allowed:

```
function singTwo(first: string, second: string) {
    console.log(`${first} / ${second}`);
}

// Logs: "Ball and Chain / undefined"
singTwo("Ball and Chain");
//      ~~~~~
// Error: Expected 2 arguments, but got 1.

// Logs: "I Will Survive / Higher Love"
singTwo("I Will Survive", "Higher Love"); // Ok

// Logs: "Go Your Own Way / The Chain"
singTwo("Go Your Own Way", "The Chain", "Dreams");
//                                           ~~~~~
// Error: Expected 2 arguments, but got 3.
```

Enforcing that required parameters be provided to a function helps enforce type safety by making sure all expected argument values exist inside the function. Failing to ensure those values exist could result in unexpected behavior in code, such as the previous `singTwo` function logging `undefined` or ignoring an argument.

Optional Parameters

Recall that in JavaScript, if a function parameter is not provided, its argument value inside the function defaults to `undefined`. Sometimes function parameters are not necessary to provide, and the intended use of the function is for that `undefined` value. We wouldn't want TypeScript to report type errors for failing to provide arguments to those op-

tional parameters. TypeScript allows annotating a parameter as optional by adding a `?` before the `:` in its type annotation—similar to optional object type properties.

Optional parameters don't need to be provided to function calls. Their types therefore always have `| undefined` added as a union type.

In the following `announceSong` function, the `singer` parameter is marked optional. Its type is `string | undefined`, and it doesn't need to be provided by callers of the function. If `singer` is provided, it may be a `string` value or `undefined`:

```
function announceSong(song: string, singer?: string) {  
    console.log(`Song: ${song}`);  
  
    if (singer) {  
        console.log(`Singer: ${singer}`);  
    }  
}  
  
announceSong("Greensleeves"); // Ok  
announceSong("Greensleeves", undefined); // Ok  
announceSong("Chandelier", "Sia"); // Ok
```

These optional parameters are always implicitly able to be `undefined`. In the previous code, `singer` starts off as being of type `string | undefined`, then is narrowed to just `string` by the `if` statement.

Optional parameters are not the same as parameters with union types that happen to include `| undefined`. Parameters that aren't marked as optional with a `?` must always be provided, even if the value is explicitly `undefined`.

The `singer` parameter in this `announceSongBy` function must be provided explicitly. It may be a `string` value or `undefined`:

```
function announceSongBy(song: string, singer: string | undefined) { /* ... */ }  
  
announceSongBy("Greensleeves");  
// Error: Expected 2 arguments, but got 1.
```

```
announceSongBy("Greensleeves", undefined); // Ok
announceSongBy("Chandelier", "Sia"); // Ok
```

Any optional parameters for a function must be the last parameters. Placing an optional parameter before a required parameter would trigger a TypeScript syntax error:

```
function announceSinger(singer?: string, song: string) {}
//                                     ~~~~
// Error: A required parameter cannot follow an optional parameter.
```

Default Parameters

Optional parameters in JavaScript may be given a default value with an `=` and a value in their declaration. For these optional parameters, because a value is provided by default, their TypeScript type does not implicitly have the `| undefined` union added on inside the function. TypeScript will still allow the function to be called with missing or `undefined` arguments for those parameters.

TypeScript's type inference works similarly for default function parameter values as it does for initial variable values. If a parameter has a default value and doesn't have a type annotation, TypeScript will infer the parameter's type based on that default value.

In the following `rateSong` function, `rating` is inferred to be of type `number`, but is an optional `number | undefined` in the code that calls the function:

```
function rateSong(song: string, rating = 0) {
    console.log(`${song} gets ${rating}/5 stars!`);
}

rateSong("Photograph"); // Ok
rateSong("Set Fire to the Rain", 5); // Ok
rateSong("Set Fire to the Rain", undefined); // Ok

rateSong("At Last!", "100");
//                                     ~~~~~
// Error: Argument of type '"100"' is not assignable
// to parameter of type 'number | undefined'.
```

Rest Parameters

Some functions in JavaScript are made to be called with any number of arguments. The `...` spread operator may be placed on the last parameter in a function declaration to indicate any “rest” arguments passed to the function starting at that parameter should all be stored in a single array.

TypeScript allows declaring the types of these rest parameters similarly to regular parameters, except with a `[]` syntax added at the end to indicate it’s an array of arguments.

Here, `singAllTheSongs` is allowed to take zero or more arguments of type `string` for its `songs` rest parameter:

```
function singAllTheSongs(singer: string, ...songs: string[]) {
  for (const song of songs) {
    console.log(`${song}, by ${singer}`);
  }
}

singAllTheSongs("Alicia Keys"); // Ok
singAllTheSongs("Lady Gaga", "Bad Romance", "Just Dance", "Poker Face"); // Ok

singAllTheSongs("Ella Fitzgerald", 2000);
//                               ~~~~
// Error: Argument of type 'number' is not
// assignable to parameter of type 'string'.
```

Return Types

TypeScript is perceptive: if it understands all the possible values returned by a function, it’ll know what type the function returns. In this example, `singSongs` is understood by TypeScript to return a `number`:

```
// Type: (songs: string[]) => number
function singSongs(songs: string[]) {
  for (const song of songs) {
    console.log(`${song}`);
  }

  return songs.length;
}
```

If a function contains multiple `return` statements with different values, TypeScript will infer the return type to be a union of all the possible returned types.

This `getSongAt` function would be inferred to return `string | undefined` because its two possible returned values are typed `string` and `undefined`, respectively:

```
// Type: (songs: string[], index: number) => string | undefined
function getSongAt(songs: string[], index: number) {
  return index < songs.length
    ? songs[index]
    : undefined;
}
```

Explicit Return Types

As with variables, I generally recommend not bothering to explicitly declare the return types of functions with type annotations. However, there are a few cases where it can be useful specifically for functions:

- You might want to enforce functions with many possible returned values always return the same type of value.
- TypeScript will refuse to try to reason through return types of recursive function.
- It can speed up TypeScript type checking in very large projects—i.e., those with hundreds of TypeScript files or more.

Function declaration return type annotations are placed after the `)` following the list of parameters.

For a function declaration, that falls just before the `{`:

```
function singSongsRecursive(songs: string[], count = 0): number {
  return songs.length ? singSongsRecursive(songs.slice(1), count + 1) : count;
}
```

For arrow functions (also known as lambdas), that falls just before the `=>`:

```
const singSongsRecursive = (songs: string[], count = 0): number =>
  songs.length ? singSongsRecursive(songs.slice(1), count + 1) : count;
```


If a `return` statement in a function returns a value not assignable to the function's return type, TypeScript will give an assignability complaint.

Here, the `getSongRecordingDate` function is explicitly declared as returning `Date | undefined`, but one of its return statements incorrectly provides a `string`:

```
function getSongRecordingDate(song: string): Date | undefined {
  switch (song) {
    case "Strange Fruit":
      return new Date('April 20, 1939'); // Ok

    case "Greensleeves":
      return "unknown";
      // Error: Type 'string' is not assignable to type 'Date'.

    default:
      return undefined; // Ok
  }
}
```

Function Types

JavaScript allows us to pass functions around as values. That means we need a way to declare the type of a parameter or variable meant to hold a function.

Function type syntax looks similar to an arrow function, but with a type instead of the body.

This `nothingInGivesString` variable's type describes a function with no parameters and a returned `string` value:

```
let nothingInGivesString: () => string;
```

This `inputAndOutput` variable's type describes a function with a `string[]` parameter, an optional `count` parameter, and a returned `number` value:

```
let inputAndOutput: (songs: string[], count?: number) => number;
```

Function types are frequently used to describe callback parameters (parameters meant to be called as functions).

For example, the following `runOnSongs` snippet declares the type of its `getSongAt` parameter to be a function that takes in an `index: number` and returns a `string`. Passing `getSongAt` matches that type, but `logSong` fails for taking in a `string` as its parameter instead of a `number`:

```
const songs = ["Juice", "Shake It Off", "What's Up"];

function runOnSongs(getSongAt: (index: number) => string) {
  for (let i = 0; i < songs.length; i += 1) {
    console.log(getSongAt(i));
  }
}

function getSongAt(index: number) {
  return `${songs[index]}`;
}

runOnSongs(getSongAt); // Ok

function logSong(song: string) {
  return `${song}`;
}

runOnSongs(logSong);
//           ~~~~~~
// Error: Argument of type '(song: string) => string' is not
// assignable to parameter of type '(index: number) => string'.
//   Types of parameters 'song' and 'index' are incompatible.
//     Type 'number' is not assignable to type 'string'.
```

The error message for `runOnSongs(logSong)` is an example of an assignability error that includes a few levels of details. When complaining that two function types aren't assignable to each other, TypeScript will typically give three levels of detail, with increasing levels of specificity:

1. The first indentation level prints out the two function types.
2. The next indentation level specifies which part is mismatched.
3. The last indentation level is the precise assignability complaint of the mismatched part.

In the previous code snippet, those levels are:

1. `logSong: (song: string) => string` is the provided type being assigned to the `getSongAt: (index: number) => string` recipient
2. The `song` parameter of `logSong` being assigned to the `index` parameter of `getSongAt`
3. `song`'s `string` type is not assignable to `index`'s `number` type

Function Type Parentheses

Function types may be placed anywhere that another type would be used. That includes union types.

In union types, parentheses may be used to indicate which part of an annotation is the function return or the surrounding union type:

```
// Type is a function that returns a union: string | undefined
let returnsStringOrUndefined: () => string | undefined;

// Type is either undefined or a function that returns a string
let maybeReturnsString: (() => string) | undefined;
```

Later parts that introduce more type syntaxes will show other places where function types must be wrapped with parentheses.

Parameter Type Inferences

It would be cumbersome if we had to declare parameter types for every function we write, including inline functions used as parameters. Fortunately, TypeScript can infer the types of parameters in a function provided to a location with a declared type.

This `singer` variable is known to be a function that takes in a parameter of type `string`, so the `song` parameter in the function later assigned to `singer` is known to be a `string`:

```
let singer: (song: string) => string;

singer = function (song) {
  // Type of song: string
  return `Singing: ${song.toUpperCase()}!`; // Ok
};
```

Functions passed as arguments to parameters with function parameter types will have their parameter types inferred as well.

For example, the `song` and `index` parameters here are inferred by TypeScript to be `string` and `number`, respectively:

```
const songs = ["Call Me", "Jolene", "The Chain"];

// song: string
// index: number
songs.forEach((song, index) => {
  console.log(`${song} is at index ${index}`);
});
```

Function Type Aliases

Remember type aliases from ? They can be used for function types as well.

This `StringToNumber` type aliases a function that takes in a `string` and returns a `number`, which means it can be used later to describe the types of variables:

```
type StringToNumber = (input: string) => number;

let stringToNumber: StringToNumber;

stringToNumber = (input) => input.length; // Ok

stringToNumber = (input) => input.toUpperCase();
//                               ~~~~~
// Error: Type 'string' is not assignable to type 'number'.
```

Similarly, function parameters can themselves be typed with aliases that happen to refer to a function type.

This `usesNumberToString` function has a single parameter which is itself the `NumberToString` aliased function type:

```
type NumberToString = (input: number) => string;

function usesNumberToString(numberToString: NumberToString) {
  console.log(`The string is: ${numberToString(1234)}`);
}
```

```
usesNumberToString((input) => `${input}! Hooray!`); // Ok

usesNumberToString((input) => input * 2);
//                                     ~~~~~
// Error: Type 'number' is not assignable to type 'string'.
```

Type aliases are particularly useful for function types. They can save a lot of horizontal space in having to repeatedly write out parameters and/or return types.

More Return Types

Now, let's look at two more return types: `void` and `never`.

Void Returns

Some functions aren't meant to return any value. They either have no `return` statements or only have `return` statements that don't return a value. TypeScript allows using a `void` keyword to refer to the return type of such a function that returns nothing.

Functions whose return type is `void` may not return a value. This `logSong` function is declared as returning `void`, so it's not allowed to return a value:

```
function logSong(song: string | undefined): void {
  if (!song) {
    return; // Ok
  }

  console.log(`${song}`);

  return true;
  // Error: Type 'boolean' is not assignable to type 'void'.
}
```

`void` can be useful as the return type in a function type declaration. When used in a function type declaration, `void` indicates that any returned value from the function would be ignored.

For example, this `songLogger` variable represents a function that takes in a `song: string` and doesn't return a

value:

```
let songLogger: (song: string) => void;

songLogger = (song) => {
  console.log(`${song}`);
};

songLogger("Heart of Glass"); // Ok
```

Note that although JavaScript functions all return `undefined` by default if no real value is returned, `void` is not the same as `undefined`. `void` means the return type of a function will be ignored, while `undefined` is a literal value to be returned. Trying to assign a value of type `void` to a value whose type instead includes `undefined` is a type error:

```
function returnsVoid() {
  return;
}

let lazyValue: string | undefined;

lazyValue = returnsVoid();
// Error: Type 'void' is not assignable to type 'string | undefined'.
```

The distinction between `undefined` and `void` returns is particularly useful for ignoring any returned value from a function passed to a location whose type is declared as returning `void`. For example, the built-in `forEach` method on arrays takes in a callback that returns `void`. Functions provided to `forEach` can return any value they want. `records.push(record)` in the following `saveRecords` function returns a `number` (the returned value from an array's `.push()`), yet is still allowed to be the returned value for the arrow function passed to `newRecords.forEach`:

```
const records: string[] = [];

function saveRecords(newRecords: string[]) {
  newRecords.forEach(record => records.push(record));
}

saveRecords(['21', 'Come On Over', 'The Bodyguard'])
```

The `void` type is not JavaScript. It's a TypeScript keyword used to declare return types of functions. Remember, it's an indication that a function's returned value isn't meant to be used, not a value that can itself be returned.

Never Returns

Some functions not only don't return a value, but aren't meant to return at all. Never-returning functions are those that always throw an error or run an infinite loop (hopefully intentionally!).

If a function is meant to never return, adding an explicit `: never` type annotation indicates that any code after a call to that function won't run. This `fail` function only ever throws an error, so it can help TypeScript's control flow analysis with type narrowing `param` to `string`:

```
function fail(message: string): never {
    throw new Error(`Invariant failure: ${message}.`);
}

function workWithUnsafeParam(param: unknown) {
    if (typeof param !== "string") {
        fail(`param should be a string, not ${typeof param}`);
    }

    // Here, param is known to be type string
    param.toUpperCase(); // Ok
}
```

NOTE

`never` is not the same as `void`. `void` is for a function that returns nothing. `never` is for a function that never returns.

Function Overloads

Some JavaScript functions are able to be called with drastically different sets of parameters that can't be represented just by optional and/or rest parameters. These functions can be described with a TypeScript syntax called *overload signatures*: declaring different versions of the function's name, parameters, and return types multiple times before one final *implementation signature* and the body of the function.

When determining whether to emit a syntax error for a call to an overloaded function, TypeScript will only look at the function's overload signatures. The implementation signature is only used by the function's internal logic.

This `createDate` function is meant to be called either with one `timestamp` parameter or with three parameters—`month`, `day`, and `year`. Calling with either of those numbers of arguments is allowed, but calling with two arguments would cause a type error because no overload signature allows for two arguments. In this example, the first two lines are the overload signatures, and the third line is the implementation signature:

```
function createDate(timestamp: number): Date;
function createDate(month: number, day: number, year: number): Date;
function createDate(monthOrTimestamp: number, day?: number, year?: number) {
    return day === undefined || year === undefined
        ? new Date(monthOrTimestamp)
        : new Date(year, monthOrTimestamp, day);
}

createDate(554356800); // Ok
createDate(7, 27, 1987); // Ok

createDate(4, 1);
// Error: No overload expects 2 arguments, but overloads
// do exist that expect either 1 or 3 arguments.
```

Overload signatures, as with other type system syntaxes, are erased when compiling TypeScript to output JavaScript.

The previous code snippet's function would compile to roughly the following JavaScript:

```
function createDate(monthOrTimestamp, day, year) {
    return day === undefined || year === undefined
        ? new Date(monthOrTimestamp)
        : new Date(year, monthOrTimestamp, day);
}
```


WARNING

Function overloads are generally used as a last resort for complex, difficult-to-describe function types. It's generally better to keep functions simple and avoid using function overloads when possible.

Call-Signature Compatibility

The implementation signature used for an overloaded function's implementation is what the function's implementation uses for parameter types and return type. Thus, each parameter in a function's overload signatures must be assignable to the parameter at the same index in its implementation signature. The overload signature's return type must also be assignable to the implementation signature's return type. In other words, the implementation signature has to be compatible with all of the overload signatures.

This `format` function's implementation signature declares its first parameter to be a `string`. While the first two overload signatures are compatible for also being type `string`, the third overload signature's `() => string` type is not compatible:

```
function format(data: string): string; // Ok
function format(data: string, needle: string, haystack: string): string; // Ok

function format(getData: () => string): string;
//      ~~~~~
// This overload signature is not compatible with its implementation signature.

function format(data: string, needle?: string, haystack?: string) {
    return needle && haystack ? data.replace(needle, haystack) : data;
}
```

In this part, you saw how a function's parameters and return types can be inferred or explicitly declared in TypeScript:

- Declaring function parameter types with type annotations
- Declaring optional parameters, default values, and rest parameters to change type system behavior
- Declaring function return types with type annotations

- Describing functions that don't return a usable value with the `void` type
- Describing functions that don't return at all with the `never` type
- Declaring function types in type annotations
- Using function overloads to describe varying function call signatures